

Initiation à la méthodologie DevOps, Gitlab CI/CD et conteneurisation Docker

Partie 5 : La Bataille des Services



Bienvenue dans votre dernière journée de devops de la semaine 🕶️

Aujourd'hui on va sortir l'arsenal pour se battre en groupe. En équipes de 3-4 personnes vous allez créer un arsenal de Services que le reste de la classe essaiera de casser... EN PLEINE DÉMO 🐱

1 - La carte de bataille (oui juste un tableau)

LIEN TABLEAU

Voilà ce qu'on voit : des carrés de couleur, une couleur par équipage.

Carré plein = le service tourne

Carré vide, il ne répond plus,

... et toute la salle le voit en même temps que vous.

Votre équipage jure ce matin de tenir un certain nombre de carrés allumés jusqu'au soir. À heure fixe, la classe aura le droit de tirer dessus, et une panne sera tirée au sort pendant que vous parlez.

Bonne nouvelle : vous savez déjà tout faire. Construire une image, la publier, la déployer par une pipeline, la surveiller, écrire la procédure qui va avec. Ce qui change aujourd'hui, c'est qu'il y a un écran, et que l'écran ne ment pas.

L'essentiel du tableau

[Le tableau] () <- insérer_adresse est une petite application que je vous ai hébergée sur [Vercel](#).

Vos services l'appellent par eux-mêmes, tout seuls, à intervalle régulier. Cet appel, c'est ce qu'on appellera un poulx : ça permettra de voir si votre flotte est vivante ou morte.

Boum boum ? (le poulx, définition) : ici ce sera un appel HTTP que votre service envoie au tableau toutes les quelques secondes pour dire "je suis vivant, voilà qui je suis".

Chaque poulx transporte cinq choses : le nom de l'équipage, sa couleur, le nom du service, le nom du conteneur qui parle, et la version d'image qui tourne. Le tableau répond, et sa réponse contient le nombre de coups que le service doit encaisser.

Ce que chaque carré raconte

On a trois états, trois histoires différentes.



PLEIN

Un poulx il y a moins de huit secondes, aucun retard.

Le service répond, et il suit le rythme.



PÂLE

Son poulx arrive encore, mais plus de quarante coups se sont accumulés.

Il répond, il ne suit plus : problème de capacité.



VIDE

Plus aucun poulx depuis huit secondes.

Il est mort, ou il n'arrive plus à parler : problème d'existence.

La nuance entre les deux derniers vaut la journée entière. Ce ne sont pas les mêmes causes, ni les mêmes réparations.

Sous les carrés de chaque équipage s'affiche son **pavillon** : une phrase libre, que vous choisissiez. Genre "On coule pas, on plie".

Cette phrase, vos services vont la lire sur leur disque et la renvoyer à chaque poulx. D'où le piège de la journée : rangée au bon endroit, elle survit au prochain déploiement. Rangée au mauvais endroit, elle s'efface de l'écran pile au moment où vous livrez, devant toute la classe.

Dernière chose que le carré affiche : le **tag de l'image** qui tourne dedans. Quand vous livrez une nouvelle version, les carrés de votre flotte changent de tag l'un après l'autre, et la classe regarde le déploiement traverser vos services en direct.

Pourquoi c'est le service qui appelle ?

Pourquoi pas l'inverse, un tableau qui irait interroger chaque service ?

Parce que le réseau l'interdit. Vos services tournent sur vos machines, derrière le réseau de l'école, souvent derrière une box et un pare-feu. Une machine extérieure ne peut pas ouvrir une connexion vers un poste sans adresse publique. C'est le mur rencontré en J3, quand un runner hébergé chez GitHub n'atteignait pas un conteneur posé sur un poste, et qu'il a fallu un runner self-hosted.

Le sens de la flèche est donc imposé par le réseau : **le trafic part toujours de chez vous vers le tableau**. Un appel sortant en HTTPS passe partout, sur le wifi de l'école comme depuis chez vous.

Retenez-le. La sixième panne de la journée sort entièrement de là.

Planning de la Journée

1. **Constitution des équipages** : la carte de compétences, les rôles, le contrat de la flotte
2. **Le tableau apparaît** au projecteur, vide, et chaque équipage annonce sa couleur, son nom et le nombre de carrés qu'il jure de tenir
3. **La construction**, tout le reste de la journée, le tableau projeté en permanence
4. **L'ouverture du feu**, à heure fixe : la classe entière tire sur toutes les flottes en même temps
5. **Les passages** : chaque équipage raconte sa flotte et encaisse une panne

2 - Constituer les équipages

Une équipe qui se forme par affinité produit toujours le même résultat : celui qui maîtrise déjà Docker fait Docker, celui qui maîtrise déjà la pipeline fait la pipeline, et chacun ressort de la journée aussi compétent qu'il y est entré. On va faire le contraire.

L'essentiel de la carte de compétences

Chacun se note de 0 à 2 sur cinq axes, sur ce qu'il a réellement fait cette semaine, pas sur ce qu'il pense avoir compris.

L'axe	0	1	2
Image et Dockerfile	j'ai suivi sans finir	mon image tourne	mon image est multi-stage, non-root, et je sais pourquoi
Compose et réseau	j'ai copié le fichier	ma stack monte	j'ai débogué un service qui n'en joignait pas un autre
Pipeline	j'ai regardé	ma pipeline est verte	j'ai fait échouer un job exprès et je l'ai réparé
Déploiement et machine cible	pas allé jusque là	mon push déploie	j'ai fait un retour arrière et je l'ai chronométré
Surveillance	pas allé jusque là	mon dashboard affiche quelque chose	mes panneaux m'ont servi à trouver une panne

Les équipages se composent sur ces cartes, avec une règle simple : **jamais deux 2 sur le même axe dans la même équipe**. Une flotte a besoin d'une compétence par rôle, pas de trois personnes qui savent faire la même chose.

Le rôle interdit

Personne ne tient le rôle sur lequel il est le plus fort. Le 2 en pipeline ne touche pas au fichier de workflow, le 2 en Docker n'écrit aucun Dockerfile. Ils font autre chose, et surtout, ils répondent aux questions.

Mais du coup c'est pas contre-productif, si le meilleur en pipeline touche pas à la pipeline ?

Ça coûte une demi-heure, et ça en rapporte deux. Expliquer un job de déploiement à quelqu'un qui ne l'a jamais écrit reste le meilleur moyen de découvrir les trois endroits où on ne l'avait pas compris. Et une équipe où une seule personne sait faire une chose tombe le jour où cette personne est absente : la passation d'astreinte de J3 mettait exactement ça en scène.

Reste à savoir qui fait quoi. Cinq rôles, cinq territoires qui ne se chevauchent pas, et chacun porte le nom de la personne qui le tient plutôt que celui de la tâche : à 15h, quand un carré s'éteint, on doit pouvoir appeler quelqu'un et pas un concept.

Ce que chacun tient

Cinq rôles à répartir. Une personne peut en tenir deux si l'équipe est à trois, mais aucun rôle ne reste vacant. Chaque phase de l'après-midi dira lequel de ces cinq membres la pilote.

Le membre d'équipe en charge des images

Cette personne décide de ce qui entre dans chaque conteneur, et surtout de ce qui n'y entre pas. Personne d'autre n'écrit de Dockerfile.

- Écrire un Dockerfile par service, en multi-stage et avec un utilisateur non privilégié, comme en J2.
- Mesurer la taille de chaque image avant et après ses décisions, puis reporter les deux chiffres dans le carnet de la flotte, le relevé chiffré que l'équipage remplit tout au long de la journée.
- Vérifier que chaque service démarre bien avec les variables d'environnement qu'on lui donne, et jamais avec une valeur écrite en dur dans l'image.
- Répondre aux questions des autres sur le contenu de leurs conteneurs, sans construire à leur place.

On sait que ce rôle est tenu quand n'importe qui dans l'équipage peut reconstruire les trois images d'une seule commande, sans rien demander.

Le membre d'équipe en charge de la livraison

Cette personne possède le fichier de workflow et la machine cible. C'est le seul territoire de la journée où on entre à une personne à la fois, parce que deux branches qui modifient la pipeline en même temps coûtent une heure à tout le monde.

- Construire et publier les images depuis la pipeline, chacune taguée avec le sha du commit.
- Envoyer le fichier de compose sur la machine cible, puis y remplacer la flotte entière sans jamais ouvrir de terminal à la main.
- Faire échouer le job de déploiement quand un service ne répond pas, plutôt que d'afficher un vert qui ment.

- Savoir revenir à la version précédente, et connaître le temps que ça prend.

On sait que ce rôle est tenu quand plus personne dans l'équipage n'a tapé une commande à la main sur la machine cible depuis la phase 4.

Le membre d'équipe en charge de l'état

Tout le reste de la flotte est jetable et se remplace à chaque déploiement. Cette personne s'occupe de la seule chose qui n'a pas le droit de disparaître : les données.

- Choisir où vivent la base de données et le pavillon, et monter les volumes qui les portent.
- Écrire la route qui reçoit le pavillon, et vérifier après chaque déploiement qu'il est toujours au tableau.
- Garder le mot de passe de la base hors du repo, dans les secrets, du premier commit jusqu'au dernier.
- Relire le travail des autres sur tout ce qui touche à une écriture sur disque.

On sait que ce rôle est tenu quand un redéploiement complet ne fait rien perdre, et que la classe le voit à l'écran.

Le membre d'équipe en charge de la mesure

Cette personne construit ce que le tableau de la classe ne donne pas : la raison. Le tableau dit qu'un carré est éteint, elle seule peut dire pourquoi.

- Exposer les métriques de chaque service, puis les brancher sur Prometheus.
- Construire les quatre panneaux du tableau de bord, et défendre chaque panneau qui reste.
- Trouver le point de bascule de la flotte, celui à partir duquel les carrés pâlisent.
- Tenir le carnet de la flotte à jour, y compris les chiffres décevants.

On sait que ce rôle est tenu quand quelqu'un nomme une panne en regardant uniquement les panneaux, sans ouvrir un terminal.

Le membre d'équipe en astreinte

C'est le rôle le plus recherché de la journée, et celui qu'on comprend en dernier. Cette personne ne construit rien. Elle garde le tableau sous les yeux en permanence, elle prévient dès qu'un carré change d'état, et elle écrit ce qui s'est passé pendant que les autres réparent. Un rôle qui a l'air vide à 14h, et qui devient le plus intense de tous dès l'ouverture du feu.

- Annoncer chaque changement au tableau, avec l'heure, sans attendre qu'on lui demande.
- Tenir le journal de bord : une entrée par panne, y compris celles que l'équipage n'a pas su réparer.
- Écrire le runbook, celui avec lequel un autre équipage remontera la flotte sans jamais vous parler.
- Prendre la parole (faire l'intro) pendant le passage devant la classe, et raconter le diagnostic à voix haute.

Note: chaque membre du groupe doit prendre la parole pour être notée.

On sait que ce rôle est tenu quand l'équipage apprend une panne par cette personne, et pas par le rire de la salle.

Deux règles s'ajoutent, pour tout le monde :

- **Personne ne fusionne sa propre pull request.** La relecture vient de quelqu'un d'un autre rôle, et c'est là que le relecteur apprend quelque chose.

- **La machine de production change de main au moins une fois.** La machine cible vit sur l'ordinateur d'un membre de l'équipage. Si votre procédure ne permet pas de remonter la flotte ailleurs que sur la machine qui l'a vue naître, ce n'est pas une procédure, c'est un souvenir.

Tout ce que je vous dis je peux vérifier si c'est vraiment respecté 😊

On pousse plus loin : les équipages à géométrie variable

Si vous êtes trois. Deux ajustements, à décider maintenant plutôt qu'à 15h :

- le rôle en astreinte se cumule avec celui de la mesure, puisque les deux regardent le même écran,
- le nombre de carrés jurés baisse d'un. Trois personnes qui promettent six services passeront la journée à réparer, et zéro minute à comprendre.

Si vous êtes cinq. Il reste une place pour un rôle que personne ne réclame et qui rapporte gros : **le saboteur**. Son après-midi consiste à casser la flotte de sa propre équipe, avant que la classe n'en ait le droit. Il tue des conteneurs au hasard, il remplit le disque, il coupe la base. Chaque problème qu'il trouve avant l'ouverture du feu est un problème que personne ne verra en public.

3 - Le contrat de la flotte



Le sujet est cadré, et c'est volontaire : le vrai sujet de la journée, c'est la chaîne de livraison, pas l'application qu'elle transporte. Autant ne pas passer une heure à chercher une idée.

L'essentiel de ce qui est imposé

Une flotte, c'est **quatre briques au minimum**, et trois d'entre elles tiennent un carré au tableau.

La brique	Ce qu'elle fait	Carré au tableau
Le front	une page web que la classe peut ouvrir et regarder	oui
L'API métier	les données de l'application, lues et écrites	oui
La base de données	PostgreSQL, ou l'équivalent de votre choix	non
Le service annexe	autre chose, qui fait un travail que les deux autres ne font pas	oui

La DB n'a pas de carré, et ce n'est pas un oubli : une base de données ne parle pas HTTP, elle ne peut pas envoyer de pouls. Elle est quand même là, bien visible, parce que le jour où elle coule, c'est le carré de votre API qui s'éteint. Un carré vide qui désigne une panne située ailleurs, c'est la première chose que la classe apprendra à lire lors du projet.

Le service annexe doit être **différent** des deux autres : compter, notifier, agréger, transformer, discuter avec l'extérieur. Un deuxième front ne compte pas, une API qui fait la même chose sur une autre table non plus.

Le contrat à annoncer

J'ai besoin que vous m'annonciez 3 choses après constitution de vos équipes :

- **son nom et sa couleur**, un code hexadécimal du type `#e05252` (deux flottes de la même teinte rendent le tableau illisible),
- **les membres de l'équipe**
- **le nombre de carrés qu'il jure de tenir** jusqu'à la fin d'après-midi, trois au minimum,
- **son pavillon**, la phrase qui s'affichera sous ses carrés.
- **son repository**

Le chiffre juré est un pari. Six carrés, c'est une flotte impressionnante à l'écran, six images à construire, six sondes à câbler et six services à garder vivants sous la charge. Trois carrés tenus proprement pendant toute l'ouverture du feu valent mieux que six dont la moitié est pâle. Chaque carré au-delà de trois compte dans l'évaluation, à condition qu'il rende un service que les autres ne rendent pas.

Votre app / Ce qui est libre

Le langage, le sujet, le nombre de services au-delà de trois. Si rien ne vient dans les cinq premières minutes, voilà une réserve où piocher, avec pour chaque ligne le front, l'API métier, puis le service annexe. Toutes ont la même qualité : la classe peut les utiliser depuis son téléphone pendant votre passage, ce qui rend votre charge réelle plutôt que simulée.

- **Un buzzer de jeu télévisé** : le gros bouton, qui a buzzé en premier, l'historique des manches
- **Un quiz en direct** : les questions et le score, les réponses et le décompte, le classement final

- **Un mur de messages éphémères** : le mur, poster et lire, le nettoyage de ce qui a expiré
- **Un compteur collectif** : le bouton et le total, l'incrément, les clics par seconde
- **Un sondage projeté** : les choix et les barres, les votes, l'export des résultats
- **Une file de karaoké** : la file, s'inscrire et passer son tour, l'alerte aux trois suivants
- **Un mini réseau social de la promo** : le fil, les posts, la détection des doublons

Le code métier de ces applications se compte en dizaines de lignes. C'est le même parti pris que depuis lundi : la difficulté n'est pas dans ce que fait l'application, elle est dans la chaîne qui l'amène en production et qui l'y maintient.

Attention, interdit de fournir une Todo API / App ou un Clickfast

Le pavillon 🏠 : le message qui doit survivre

Le pavillon se hisse par un appel HTTP sur un de vos services, qui l'écrit sur son disque et le renvoie ensuite à chaque pouls.

Ce jeu du message, c'est un exercice de persistance. Un fichier écrit dans le système de fichiers d'un conteneur vit exactement aussi longtemps que ce conteneur : le prochain déploiement le remplace, et le fichier part avec lui. Un fichier écrit dans un [volume Docker](#), un espace de stockage géré par Docker et indépendant du cycle de vie des conteneurs, survit au remplacement. On l'a vu en J1 avec les données de la Todo API. Aujourd'hui l'enjeu est public : si le pavillon tombe pendant que vous livrez, la classe le voit avant vous.

FAQ

Le front doit-il tomber quand l'API tombe ?

Non, et c'est le réflexe le plus précieux de la journée.

Imaginez : votre API métier meurt, son carré s'éteint. Que fait le front ? Dans l'écriture la plus naturelle, celle qu'on produit tous en premier, il appelle l'API au chargement de la page, l'appel échoue, une exception remonte, et la page ne s'affiche pas. Deux carrés éteints pour une seule panne. Là le tableau raconterait une histoire plus grave que la réalité.

La dégradation gracieuse, c'est quoi ? Un service qui continue de rendre la partie de son travail qui ne dépend pas de ce qui est tombé, au lieu de tomber avec.

Un front qui dégrade proprement affiche sa page et ses éléments statiques, et remplace la zone qui dépend de l'API par un message honnête du type "données indisponibles". Son carré reste plein, parce que lui, il répond.

La question vaut pour chaque service de la flotte : *si tout ce dont je dépends disparaît, est-ce que je réponds encore ?* Un service dont la réponse est non n'est pas cassé, mais il propage les pannes des autres, et sa propre santé ne veut plus rien dire.

4 - Travailler à plusieurs sur un même repo

Depuis lundi, chacun pousse sur `main` sans se poser de question. Aujourd'hui vous êtes trois ou quatre sur le même repo, et une pull request mal gérée coûte une heure à tout le monde.

Le repo de groupe, et qui le crée

Personne ne vous fournit de repo aujourd'hui : c'est l'équipage qui crée le sien, et ça prend cinq minutes. Il ne part pas de zéro pour autant, puisqu'on récupère le travail de la veille de l'un d'entre vous, avec son Dockerfile de production, sa pipeline qui déploie, son compose et sa procédure.

Le membre d'équipe en charge de la livraison s'en occupe, dans cet ordre :

1. Créez un repo vide sur GitHub, au nom de l'équipage, sans README ni `.gitignore` : le moindre fichier créé là évite un conflit inutile au premier push.
2. Poussez-y le repo de la veille de l'un des membres, celui qui déployait déjà en J3.
Attention, je vous demande de ne pas reposer les mêmes fonctionnalités et les mêmes services, sinon c'est trop facile.
3. Ajoutez les autres membres comme collaborateurs, avec le droit d'écriture, dans `Settings` puis `Collaborators`.
4. Protégez `main` si votre plan GitHub le permet, pour que la fusion passe forcément par une pull request.

```
# Depuis le repo de la veille, sur l'ordinateur du membre en charge de la livraison
git remote add equipage git@github.com:<compte>/<nom-de-l-equipage>.git
git push equipage main

# Chacun des autres membres clone le repo de l'équipage, et travaille uniquement dedans
git clone git@github.com:<compte>/<nom-de-l-equipage>.git
```

Devrait afficher le même historique de commits chez tout le monde, celui de la semaine, avec le dernier commit de J3 en haut.

Ensuite, quatre règles, celles du [GitHub flow](#), la façon la plus répandue de travailler à plusieurs sur un dépôt Git :

- **main est toujours déployable.** Ce qui est sur `main` est ce qui tourne sur la machine cible.
- **Un travail, une branche**, nommée par ce qu'elle fait : `front-degradation`, `pipeline-deploy`, `service-stats`.
- **Un retour sur main, une pull request**, relue par quelqu'un d'un autre rôle, jamais par son auteur.
- **Des pull requests petites et fréquentes**, une par intention. À cinquante fichiers, elle n'est plus relue, elle est approuvée les yeux fermés.

Une pull request, en une ligne : une demande de fusionner une branche dans une autre, qui ouvre un espace de discussion et de relecture avant que le code ne rejoigne le tronc commun.

Le conflit qui coûte une soirée

Les conflits de fusion n'arrivent presque jamais sur le code métier. Ils arrivent sur les trois fichiers que tout le monde touche : le compose, le workflow de la pipeline, le fichier d'environnement d'exemple.

Trois habitudes suffisent :

- **Un fichier partagé, une personne à la fois.** Celui qui modifie le compose l'annonce, le fait, et le pousse dans la demi-heure.

- Un `git pull --rebase origin main` avant chaque pull request. Le conflit se règle sur votre branche, par la personne qui connaît son propre code.
- Le compose se découpe en blocs, un service par bloc, chacun le sien. Deux personnes qui ajoutent leur service à la fin du fichier créent un conflit ; deux personnes qui écrivent dans leur bloc n'en créent aucun.

5 - À votre tour - Tenir sa flotte

Le projet

À la fin de la journée, le repo de groupe et la machine cible racontent ceci :

- **trois services minimum plus une base**, chacun avec sa propre image publiée sur un registry,
- un `git push` sur `main` qui redéploie la flotte entière sur la machine cible, sans une seule commande tapée à la main,
- **trois carrés allumés en permanence** sur le tableau de la classe, avec le pavillon de l'équipage dessous,
- **un pavillon qui survit au redéploiement**, parce qu'il vit dans un volume et pas dans un conteneur,
- **une flotte qui encaisse les salves sans pâlir**, parce que la capacité a été mesurée et augmentée,
- **six pannes essayées et documentées**, avec pour chacune ce qui s'est affiché au tableau et la manœuvre qui répare,
- **un tableau de bord qui explique le cas où un carré s'éteint** plutôt que de se contenter de le constater,
- **une procédure remontant votre flotte sur une autre machine**, suivable par quelqu'un d'un autre équipage.

L'objectif tient en une phrase : *quand un carré s'éteint, quelqu'un dans l'équipage sait dire pourquoi avant de savoir comment le rallumer.*

Plusieurs phases y mènent, regroupées en sept paliers. Chacune s'ouvre sur ses quatre lignes de cadrage, à lire avant de toucher un clavier : le premier réflexe d'une phase n'est pas d'ouvrir un fichier, il est de se mettre d'accord dessus.

Palier 1 : le premier carré s'allume

Phase 1 : le repo de groupe et la machine cible

- **Objectif** : avoir une machine cible qui répond, et un repo où quatre personnes peuvent pousser sans se marcher dessus.
- **Point de départ** : le repo d'un membre de l'équipage, celui qui déployait déjà en J3, et sa maquette `vm-prod`.
- **Qui pilote** : le membre d'équipe en charge de la livraison.
- **Terminé quand** : tout le monde a poussé une branche sur le repo de groupe, et `docker ps` répond en SSH sur `vm-prod`.

Avant de commencer : retenez que le repo se note sur ses commits autant que sur son résultat. Faites un commit par phase terminée, ajoutez les fichiers un par un, jamais de `git add .` en aveugle, et ouvrez une branche par intention.

Créez d'abord le repo de l'équipage, en suivant les quatre étapes vues plus haut, et vérifiez que chacun peut cloner et pousser dessus avant d'aller plus loin. Une personne qui découvre à 15h qu'elle n'a pas les droits d'écriture, c'est une pull request qui attend.

La machine cible est celle de J3, la maquette `vm-prod` : un conteneur qui embarque un serveur SSH et son propre Docker, et qui joue le rôle du serveur de production. Elle vit sur l'ordinateur du membre d'équipe en charge de la livraison. Si le conteneur a été arrêté hier, il se rallume :

```
# La machine cible d'avant-hier, telle qu'on l'a laissée
docker start vm-prod

# On vérifie qu'elle répond toujours, et que son Docker interne est vivant
ssh -i deploy_key -p 2222 root@localhost 'docker ps'
```

Devrait afficher la liste des conteneurs qui tournaient dessus, éventuellement vide, mais sans erreur de connexion.

Si `vm-prod` a disparu, ou si elle vivait sur le poste de quelqu'un d'autre, elle se reconstruit à partir du `Dockerfile.vm` de J3, resté dans le repo. C'est le premier test réel de votre procédure de la veille : si elle ne suffit pas à remonter la machine cible, c'est maintenant qu'on la corrige, pas à 16h.

Phase 2 : un service, et son pouls

- **Objectif** : allumer le premier carré de votre flotte au tableau, devant toute la classe.
- **Point de départ** : le repo de groupe de la phase 1, et le service le plus simple de votre application.
- **Qui pilote** : le membre d'équipe en charge des images, avec celui en astreinte à côté pour lire le tableau.
- **Terminé quand** : le carré apparaît en moins de dix secondes, s'éteint quand on coupe le service, et se rallume seul après une coupure réseau.

Le réflexe repo : faites un commit à la seconde où le carré s'allume, avant de toucher à quoi que ce soit d'autre.

Un seul service pour commencer, le plus simple à faire tourner. Il doit répondre sur une route de santé, et envoyer son pouls au tableau.

Voilà le code du pouls. Étant donné que c'est de la plomberie, et que la réécrire coûterait une heure par équipage pour un résultat strictement identique, je vous le donne, c'est cadeau. Le voici en Node.js, à déposer dans le repo et à importer depuis chaque service :

```
// pouls.js : à importer depuis chaque service qui doit tenir un carré au tableau
import os from "node:os";
import fs from "node:fs";

const TABLEAU = process.env.TABLEAU_URL;
const GROUPE = process.env.GROUPE;
const COULEUR = process.env.COULEUR || "#888888";
const SERVICE = process.env.SERVICE;
```

```

const VERSION = process.env.VERSION || "dev";
const PAVILLON = process.env.PAVILLON_FICHER || "/data/pavillon.txt";
const MOI = process.env.URL_INTERNE || "http://localhost:3000";

let totalEncaisse = 0; // depuis le démarrage de ce process, affiché sur le carré
let aDeclarer = 0; // encaissés depuis le dernier pouls, ce que le tableau attend

// Le pavillon est relu du disque à chaque pouls : s'il vit dans un volume, il
// traverse le redéploiement, sinon il disparaît du tableau devant toute la classe.
function lirePavillon() {
  try {
    return fs.readFileSync(PAVILLON, "utf8").trim();
  } catch {
    return "";
  }
}

// Un coup, c'est une vraie requête sur sa propre route de travail. Les coups partent
// par paquets de dix en parallèle, sinon un gros retard bloquerait le pouls suivant.
async function encaisser(nombre) {
  const aFaire = Math.min(nombre, 300);
  for (let debut = 0; debut < aFaire; debut += 10) {
    const paquet = [];
    for (let i = debut; i < Math.min(debut + 10, aFaire); i++) {
      paquet.push(
        fetch(`${MOI}/travail`)
          .then((reponse) => {
            if (reponse.ok) {
              totalEncaisse++;
              aDeclarer++;
            }
          })
          .catch(() => {}))
      );
    }
    await Promise.all(paquet);
  }
}

async function envoyerPouls() {
  let attente = 5000;
  const declares = aDeclarer;
  try {
    const reponse = await fetch(`${TABLEAU}/api/pouls`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        groupe: GROUPE,
        couleur: COULEUR,
        service: SERVICE,
        pod: os.hostname(),
        version: VERSION,
        pavillon: lirePavillon(),
      })
    });
  }
}

```

```

        encaisses: declares,
        total_encaisse: totalEncaisse,
    )),
    });
    // Les coups déclarés ne sont retirés du compteur local qu'une fois le tableau
    // au courant : si l'appel échoue, ils repartiront dans le pouls suivant.
    aDeclarer -= declares;
    const ordre = await reponse.json();
    attente = ordre.prochain_pouls_ms || 5000;
    if (ordre.coups_a_encaisser > 0) await encaisser(ordre.coups_a_encaisser);
} catch (erreur) {
    console.error("[pouls] tableau injoignable :", erreur.message);
}
setTimeout(envoyerPouls, attente);
}

export function demarrerLePouls() {
    if (!TABLEAU || !GROUPE || !SERVICE) {
        console.error("[pouls] TABLEAU_URL, GROUPE et SERVICE sont obligatoires");
        return;
    }
    console.log(`[pouls] ${GROUPE}/${SERVICE} vers ${TABLEAU}`);
    envoyerPouls();
}

```

Pour une flotte en Python, le même pouls, sans aucune dépendance en dehors de la bibliothèque standard :

```

# pouls.py : demarrer_le_pouls() lance un thread, à appeler une fois au démarrage
import json, os, socket, threading, time, urllib.error, urllib.request

TABLEAU = os.environ.get("TABLEAU_URL", "")
GROUPE = os.environ.get("GROUPE", "")
COULEUR = os.environ.get("COULEUR", "#888888")
SERVICE = os.environ.get("SERVICE", "")
VERSION = os.environ.get("VERSION", "dev")
PAVILLON = os.environ.get("PAVILLON_FICHER", "/data/pavillon.txt")
MOI = os.environ.get("URL_INTERNE", "http://localhost:8000")

_total_encaisse = 0
_a_declarer = 0

def _lire_pavillon():
    try:
        with open(PAVILLON, encoding="utf-8") as fichier:
            return fichier.read().strip()
    except OSError:
        return ""

def _appeler(url, donnees=None, delai=4):
    corps = None if donnees is None else json.dumps(donnees).encode("utf-8")
    requete = urllib.request.Request(url, data=corps, method="POST" if corps else "GET")

```



```

if corps:
    requete.add_header("Content-Type", "application/json")
    with urllib.request.urlopen(requete, timeout=delai) as reponse:
        return reponse.read()

def _encaisser(nombre):
    global _total_encaisse, _a_declarer
    for _ in range(min(nombre, 300)):
        try:
            _appeler(f"{MOI}/travail")
            _total_encaisse += 1
            _a_declarer += 1
        except (urllib.error.URLError, socket.timeout, OSError):
            return

def _pouls():
    global _a_declarer
    while True:
        attente = 5.0
        declares = _a_declarer
        try:
            brut = _appeler(f"{TABLEAU}/api/pouls", {
                "groupe": GROUPE, "couleur": COULEUR, "service": SERVICE,
                "pod": socket.gethostname(), "version": VERSION,
                "pavillon": _lire_pavillon(),
                "encaisses": declares, "total_encaisse": _total_encaisse,
            })
            # Retirés du compteur local seulement une fois le tableau au courant.
            _a_declarer -= declares
            ordre = json.loads(brut)
            attente = ordre.get("prochain_pouls_ms", 5000) / 1000
            if ordre.get("coups_a_encaisser", 0) > 0:
                _encaisser(ordre["coups_a_encaisser"])
        except (urllib.error.URLError, socket.timeout, OSError, ValueError) as erreur:
            print(f"[pouls] tableau injoignable : {erreur}", flush=True)
        time.sleep(attente)

def demarrer_le_pouls():
    if not (TABLEAU and GROUPE and SERVICE):
        print("[pouls] TABLEAU_URL, GROUPE et SERVICE sont obligatoires", flush=True)
        return
    threading.Thread(target=_pouls, daemon=True).start()

```

Ce que le pouls attend de vous, en revanche, reste à écrire :

- une route `/travail` sur chaque service qui envoie son pouls, qui fait un petit travail réel et répond ; elle encaissera les coups au palier 4, mais elle doit exister dès maintenant, même vide,
- les **variables d'environnement** `TABLEAU_URL`, `GROUPE`, `COULEUR`, `SERVICE`, `VERSION` et `URL_INTERNE`, injectées par le compose et jamais écrites en dur dans l'image,

- l'appel à `démarrerLePouls()` au démarrage du service.

Le contrôle avant de passer à la suite. Trois vérifications, dans cet ordre :

1. Le service tourne en local et sa route de santé répond. Le carré doit apparaître au tableau en moins de dix secondes.
2. On coupe le service. Le carré doit disparaître dans les huit secondes, pas rester allumé.
3. On coupe le wifi vingt secondes, puis on le rétablit sans toucher au service. Le carré s'éteint, puis se rallume tout seul. Un pouls qui abandonne définitivement après une erreur réseau est un pouls à réparer, et il vaut mieux le voir ici qu'au pire moment.

Palier 2 : la flotte au complet, livrée par un push

Un carré, c'est un service qui tourne sur un poste. Une flotte, c'est plusieurs services qui tiennent ensemble sur une machine que personne n'a sous les yeux, et que seule la pipeline touche. Ce palier fait la bascule.

Phase 3 : les autres services, une image chacun

- **Objectif** : faire tourner trois services distincts, avec trois images et trois cycles de vie indépendants.
- **Point de départ** : un service qui envoie déjà son pouls, et le reste de l'application encore à écrire.
- **Qui pilote** : le membre d'équipe en charge des images, avec une pull request par service.
- **Terminé quand** : la flotte monte en local avec `compose.prod.yml`, trois carrés au tableau, et aucun service ne démarre en silence avec une configuration incomplète.

Rappel notation : ouvrez une pull request par service, relue par quelqu'un d'un autre rôle. C'est aussi le moment où l'équipe découvre si ses trois services se ressemblent trop.

Les services restants s'écrivent maintenant : le front, l'API métier, le service annexe, et la base qui vient d'une image officielle sans être reconstruite. Chaque service a **son propre Dockerfile** et **sa propre image**, taguée avec le sha du commit comme depuis J2.

Le piège du palier se paie cher : une image unique qui contiendrait les trois services, lancée avec trois commandes différentes. Ça marche, et ça détruit tout ce qu'on a construit cette semaine, puisqu'une correction sur le front oblige alors à reconstruire et redéployer l'API. Un service, une image, un cycle de vie.

Le fichier de compose de production rassemble le tout. Sa structure, avec le premier bloc écrit et les autres à compléter :

```
# compose.prod.yml : la flotte telle qu'elle tourne sur la machine cible
services:
  front:
    image: ${REGISTRY}/${GROUPE}-front:${TAG}
    restart: unless-stopped
    environment:
      TABLEAU_URL: ${TABLEAU_URL}
      GROUPE: ${GROUPE}
      COULEUR: ${COULEUR}
      SERVICE: front
      VERSION: ${TAG}
      URL_INTERNE: http://front:3000
      API_URL: http://api:3000
```

```

ports:
  # 3000 est un des quatre ports que la machine cible publie depuis J3,
  # avec 9090 pour Prometheus, 3001 pour Grafana et 2222 pour SSH
  - "3000:3000"
depends_on:
  - api

api:
  # TODO : même forme que front, avec SERVICE: api et sa connexion à la base
  # Attention : pas de ports exposés vers l'extérieur si seul le front l'appelle

annexe:
  # TODO : votre troisième carré

db:
  image: postgres:16-alpine
  restart: unless-stopped
  environment:
    POSTGRES_PASSWORD: ${DB_PASSWORD}
  volumes:
    - donnees:/var/lib/postgresql/data

volumes:
  donnees:

```

Notez `restart: unless-stopped` sur chaque service : c'est la [politique de redémarrage de Docker](#), qui demande au démon de relancer un conteneur qui s'arrête tout seul. Elle décidera du sort de la première panne de la journée, alors autant la poser en connaissance de cause plutôt que par habitude.

Ce qu'on vérifie ici. La flotte monte en local avec `docker compose -f compose.prod.yml up`, et les trois carrés apparaissent au tableau. Puis deux cas moins agréables : la stack lancée sans `TABLEAU_URL`, où les services doivent démarrer quand même en le signalant dans leurs logs, et la stack lancée sans le mot de passe de la base. Un service qui démarre en silence avec une configuration incomplète est bien plus dangereux qu'un service qui refuse de démarrer.

Phase 4 : le push qui livre la flotte entière

- **Objectif** : remplacer la flotte entière sur la machine cible d'un seul `git push`, sans une seule commande tapée à la main.
- **Point de départ** : les trois images publiées et le `compose.prod.yml` de la phase 3, plus la pipeline de J3 qui déployait un service.
- **Qui pilote** : le membre d'équipe en charge de la livraison, seul dans le fichier de workflow.
- **Terminé quand** : un push anodin change les tags des carrés sans en éteindre un, et le même push rejoué deux fois de suite ne casse rien.

Avant de coder : rappelez-vous que cette phase touche au fichier de workflow, celui que tout le monde veut modifier. Entrez-y à une personne à la fois, et ne laissez pas la branche ouverte une heure.

La pipeline de J3 déployait un service. Elle doit maintenant en déployer trois ou plus, et trois choses changent :

- **construire plusieurs images** dans le même workflow, idéalement en parallèle, chacune taguée avec le même sha de commit,

- **envoyer le fichier de compose** sur la machine cible en même temps que le reste, parce que c'est lui qui décrit la flotte,
- **remplacer la flotte entière** d'un seul geste, sur la machine cible, avec un `docker compose pull` suivi d'un `docker compose up -d`.

Le squelette du job de déploiement, avec les trous à combler :

```
# .github/workflows/deploy.yml, extrait : le job qui livre la flotte
deploy:
  needs: [build]          # les images doivent exister avant qu'on aille les chercher
  runs-on: self-hosted    # le runner de J3, celui qui atteint la machine cible
  steps:
    - uses: actions/checkout@v4

    - name: Envoyer la description de la flotte
      run: |
        # TODO : copier compose.prod.yml sur la machine cible, via scp, dans /srv/flotte
        # La clé privée vient des secrets du repo, jamais du repo lui-même

    - name: Remplacer la flotte
      run: |
        # TODO : en SSH sur la machine cible, depuis /srv/flotte
        # 1. écrire le fichier .env, avec le sha du commit comme TAG
        #    (la variable github.sha du workflow, comme en J2)
        # 2. docker compose pull
        # 3. docker compose up -d
        # Cette séquence doit pouvoir être relancée deux fois de suite sans rien casser

    - name: Vérifier que la flotte est debout
      run: |
        # TODO : interroger la route de santé de chaque service exposé
        # Le job doit échouer si un service ne répond pas au bout de trente secondes
```

Cette dernière étape n'est pas décorative. Une pipeline verte qui a livré une flotte morte est pire qu'une pipeline rouge, parce qu'elle vous fait chercher ailleurs. Même principe que le job qui vérifiait le rollout en J4.

Trois scénarios à rejouer avant de continuer :

- Un push anodin sur `main`, une virgule dans le front. Les carrés doivent changer de tag l'un après l'autre, sans qu'aucun ne s'éteigne plus de quelques secondes.
- Le même push, lancé deux fois de suite. Le deuxième passage ne doit rien casser : c'est l'idempotence de J3, sur une flotte au lieu d'un service.
- Un push qui contient une image cassée, par exemple une commande de démarrage volontairement fausse. La pipeline doit devenir rouge à l'étape de vérification, et le carré concerné doit s'éteindre. Ce cas resservira au palier 5.

Palier 3 : ce que le remplacement d'un conteneur emporte avec lui

Jusqu'ici, chaque déploiement remplace des conteneurs qui ne retiennent rien. Ce palier ajoute la seule chose qu'un conteneur ne doit jamais perdre.

Phase 5 : hisser le pavillon, et le garder

- **Objectif** : afficher le pavillon sous vos carrés, et l'y garder après un déploiement.
- **Point de départ** : la flotte livrée par la pipeline, et une route `/pavillon` qui n'existe pas encore.
- **Qui pilote** : le membre d'équipe en charge de l'état, celui des images en relecture.
- **Terminé quand** : le pavillon hissé est toujours au tableau quinze secondes après un vrai push.

Un mot sur le repo : ouvrez deux branches et deux pull requests, puisque cette phase touche au compose et à un service. La relecture croisée fait la jonction entre les deux.

Une route sur un de vos services reçoit le pavillon et l'écrit sur le disque, le pouls le renvoie ensuite au tableau. La route est simple, l'enjeu est ailleurs :

```
// Le pavillon arrive par POST, part sur le disque, et le pouls le relit ensuite
app.post("/pavillon", (requete, reponse) => {
  // TODO : écrire le corps de la requête dans le fichier PAVILLON_FICHIER
  // Le dossier parent doit exister, et la longueur est limitée à 140 caractères
  // TODO : répondre 201, et 400 si le message est vide
});
```

Puis vient la vraie question : *où ce fichier est-il écrit ?* Trois emplacements possibles, et un seul survit.

Où le pavillon est écrit	Survit à un redémarrage du conteneur	Survit à un redéploiement
Dans le système de fichiers du conteneur	oui	non
Dans un volume nommé, monté sur le service	oui	oui
Dans la base de données	oui	oui, si la base a son propre volume

Les deux dernières lignes sont des choix défendables, la première est un piège qui se referme en public.

Le test qui compte, en trois gestes : hisser le pavillon, le voir au tableau, puis déclencher un vrai redéploiement par un push. S'il est toujours là quinze secondes plus tard, le palier est tenu. S'il a disparu, l'emplacement était mauvais, et c'était exactement ce qu'il fallait découvrir maintenant.

Phase 6 : des sondes qui disent la vérité

- **Objectif** : écrire une route de santé capable de dire que la base est tombée, au lieu de dire que le serveur HTTP tourne.
- **Point de départ** : les routes de santé écrites en J1, qui répondent `200` sans rien vérifier du tout.

- **Qui pilote** : le membre d'équipe en charge de la mesure, celui de l'état pour les dépendances.
- **Terminé quand** : base arrêtée à la main, l'API passe `unhealthy` en moins de trente secondes pendant que le carré du front reste plein.

Côté commits : faites un commit par service instrumenté, pour pouvoir revenir en arrière service par service quand une sonde se révèle trop stricte.

Chaque service a une route de santé depuis J1. Reste à savoir ce qu'elle regarde vraiment. Une route qui répond `200` parce que le serveur HTTP tourne ne prouve rien d'autre que le fait que le serveur HTTP tourne, et J4 a montré ce que ça coûte : un service déclaré sain pendant que sa base est injoignable.

Deux niveaux à distinguer, sur chaque service :

- **Le service répond** : le process est vivant, il accepte les connexions. C'est ce que le pouls prouve déjà, et c'est ce que le carré affiche.
- **Le service fonctionne** : ses dépendances répondent, il peut faire son travail. C'est ce que la route de santé doit vérifier, en interrogeant vraiment la base ou le service dont elle dépend.

Le [healthcheck de Docker](#) branche la deuxième au niveau de la stack, ce qui rend l'état visible sans ouvrir un navigateur :

```
healthcheck:
  test: ["CMD", "wget", "-qO-", "http://localhost:3000/sante"]
  interval: 10s
  timeout: 3s
  retries: 3
  start_period: 15s
```

Ce qui doit se produire, et ce qui ne doit pas :

- La base est arrêtée à la main. La route de santé de l'API doit passer en erreur en moins de trente secondes, et `docker compose ps` doit afficher le service comme `unhealthy`.
- Pendant ce temps, le carré du front doit rester plein, et le front doit afficher sa page avec sa zone de données remplacée par un message honnête.
- La base est relancée. Tout doit revenir sans qu'aucune commande ne soit tapée sur les services, et sans redéploiement.

Palier 4 : encaisser les salves

Trois carrés pleins sur une mer d'huile, c'est le contrat rempli. Ce palier prépare le moment où trente personnes cliqueront dessus en même temps.

Phase 7 : la route de travail, et la première saturation

- **Objectif** : faire encaisser de vrais coups à vos services, et savoir à partir de combien ils pâlisent.
- **Point de départ** : la route `/travail` créée vide au palier 1, qui répond sans rien faire.
- **Qui pilote** : le membre d'équipe en charge de l'état pour le travail réel, celui de la mesure pour les chiffres.
- **Terminé quand** : une salve tirée depuis votre poste fait pâlir le carré, qui redevient plein tout seul ensuite.

Avant de lancer : rangez les chiffres que produit cette phase dans le repo, dans le journal de bord, et pas dans un fichier local que personne ne reverra.

La route `/travail` créée au palier 1 doit maintenant faire quelque chose de réel. Un coup, c'est une requête sur cette route, et elle doit coûter un peu : lire en base, calculer, écrire. Quelques millisecondes suffisent, l'important est que ce soit du vrai travail et pas un `return 200` immédiat.

```
// La route qui encaisse les coups : elle doit faire un travail réel et mesurable
app.get("/travail", async (requete, reponse) => {
  // TODO : un travail qui coûte quelques millisecondes et qui touche à la réalité
  //       du service (une lecture en base, un calcul, une écriture)
  // TODO : répondre 200 si le travail a abouti, 503 s'il n'a pas pu être fait
});
```

Le tableau se charge du reste : quand la classe clique, il transmet les coups à votre service au pouls suivant, et votre service déclare combien il en a absorbés. Le retard s'affiche sur le carré.

Le moment intéressant est celui où ça ne suit plus. Une salve tirée depuis votre propre poste permet de trouver le point de bascule :

```
# Une salve manuelle sur son propre service, pour trouver où ça casse.
# Le feu doit être ouvert au tableau, sinon la salve est ignorée.
for i in $(seq 1 5); do
  curl -s -X POST "$TABLEAU_URL/api/coups" \
    -H 'Content-Type: application/json' \
    -d "{\"groupe\":\"$GROUPE\",\"service\":\"api\",\"nombre\":200}"
done
```

Devrait afficher, au tableau, un carré qui pâlit d'abord, puis qui redevient plein au fur et à mesure que le retard se résorbe.

Phase 8 : plus d'exemplaires, et le carnet de la flotte

- **Objectif :** encaisser davantage avec la même flotte, parce qu'on a mesuré avant de dupliquer.
- **Point de départ :** une route `/travail` qui coûte quelques millisecondes, et le point de bascule trouvé en phase 7.
- **Qui pilote :** le membre d'équipe en charge de la mesure, celui de la livraison pour le passage à l'échelle.
- **Terminé quand :** le carnet de la flotte a ses six lignes remplies, chacune avec son avant et son après.

Petit rappel : prenez vos mesures avant et après, jamais seulement après. Un chiffre sans point de comparaison ne prouve rien.

Un seul exemplaire d'un service traite les coups les uns après les autres. Deux exemplaires derrière le même nom traitent deux fois plus vite, à condition que rien dans le service ne suppose qu'il est seul au monde. En Docker Compose, ça se demande en une option :

```
# Trois exemplaires de l'API, derrière le même nom de service sur le réseau interne
docker compose -f compose.prod.yml up -d --scale api=3
```


Deux pièges vous attendent, et les rencontrer fait partie du travail : un service qui expose un port fixe vers l'extérieur ne peut pas être dupliqué, puisque deux conteneurs ne peuvent pas écouter sur le même port de la machine. Et un service qui garde son état en mémoire donne des réponses différentes selon l'exemplaire qui répond.

Le **carnet de la flotte** rassemble les mesures de la journée dans le repo. Il s'ouvre ici, et se complète aux paliers suivants :

Ce qu'on mesure	Avant	Après	Ce qui a changé entre les deux
Taille de chaque image			
Durée entre le push et le dernier carré à jour			
Coups encaissés par poulx, avec un exemplaire			
Coups encaissés par poulx, avec trois exemplaires			
Nombre de coups avant que le carré ne pâlisce			
Temps de retour à un carré plein après une salve de mille coups			

Est-ce que trois exemplaires encaissent vraiment trois fois plus ? La réponse est rarement oui, et la raison de l'écart est plus instructive que le chiffre lui-même. La base de données, elle, n'a pas été dupliquée.

Palier 5 : les six pannes

Une flotte qui tient par beau temps n'a rien prouvé. Ce palier est celui où on la saborde soi-même, méthodiquement, avant que quelqu'un d'autre ne le fasse en public.

Phase 9 : le tirage, et la manœuvre

- **Objectif** : vivre six pannes chez vous, en privé, avant que la classe n'ait le droit de les provoquer en public.
- **Point de départ** : la flotte complète sur la machine cible, et `pannes.sh` déposé dans le repo.
- **Qui pilote** : le membre d'équipe en astreinte, avec tout l'équipage autour du tableau.
- **Terminé quand** : chaque panne est tirée au moins une fois, chacune a son entrée au journal, et trois réparations sont chronométrées.

Sur le repo : écrivez une entrée de journal de bord pour chaque panne essayée, même celles qu'on n'a pas su réparer. Surtout celles-là.

Six pannes, une par carte. Le script suivant en tire une au hasard et l'exécute sur la machine cible, sans dire laquelle :

```
#!/usr/bin/env bash
# pannes.sh : tire une panne au hasard et l'applique sur la machine cible.
# À lancer depuis votre poste, la machine cible doit être joignable en SSH.
# Personne dans l'équipage ne regarde le numéro qui sort, c'est tout l'intérêt.
set -u
```

```

CIBLE="ssh -i deploy_key -p 2222 root@localhost"
SERVICES=(front api annexe)
VICTIME=${SERVICES[$RANDOM % ${#SERVICES[@]}]}
ID="docker ps -qf name=$VICTIME | head -1"

case $((RANDOM % 6)) in
  0) $CIBLE "docker kill \${$ID}" ;;
  1) $CIBLE "docker stop \$(docker ps -qf name=db | head -1)" ;;
  2) $CIBLE "docker exec \${$ID} chmod 000 /data" ;;
  3) $CIBLE "cd /srv/flotte && sed -i 's|^DB_PASSWORD=.*|DB_PASSWORD=|' .env && docker
compose up -d api" ;;
  4) $CIBLE "cd /srv/flotte && sed -i 's|^TAG=.*|TAG=nexistepas|' .env && docker compose
up -d $VICTIME" ;;
  5) $CIBLE "cd /srv/flotte && sed -i 's|^TABLEAU_URL=.*|TABLEAU_URL=http://127.0.0.1:1|'
.env && docker compose up -d $VICTIME" ;;
esac
echo "Le tableau va parler. Qu'est-ce qui s'est éteint, et pourquoi ?"

```

Chaque panne se travaille en trois temps : **ce que le tableau montre, ce qui a réellement cassé, la manœuvre qui répare**. Les deux premières colonnes ne se devinent pas depuis le script, elles se lisent sur l'écran et dans les logs.

La panne	Ce que la classe voit au tableau	Ce qui a cassé
1. Le conteneur tué	un carré s'éteint, puis revient tout seul, ou pas	le process est mort, et la politique de redémarrage décide de la suite
2. La base coupée	le carré de l'API pâlit ou s'éteint, celui du front dépend de votre dégradation	rien n'est cassé dans l'API, sa dépendance a disparu
3. Le pavillon muet	le pavillon disparaît, tous les carrés restent pleins	le dossier du volume n'est plus lisible, le service tourne pourtant très bien
4. Le secret effacé	un carré s'éteint au redémarrage suivant	la configuration est incomplète, l'image est intacte
5. La version introuvable	un carré s'éteint et ne revient pas	le tag demandé n'existe pas sur le registry
6. Le tableau injoignable	un carré s'éteint alors que le service va parfaitement bien	rien du tout, sauf le chemin entre votre service et le tableau

La sixième est la plus retorse, et elle boucle sur ce qu'on disait ce matin : le tableau ne dit pas si votre service va bien, il dit si votre service **arrive à raconter** qu'il va bien. Quelqu'un qui ouvre l'application dans son navigateur pendant que le carré est éteint le découvre en dix secondes ; quelqu'un qui ne regarde que le tableau cherche vingt minutes.

La vérification du palier : chaque panne tirée au moins une fois, chacune avec son entrée dans le journal, et trois réparations chronométrées au minimum. Un tirage qui ne casse rien de visible compte aussi et mérite sa ligne : une panne invisible est un trou de surveillance, et le palier 6 existe pour ça.

Phase 10 : le runbook de la flotte

- **Objectif** : écrire un document avec lequel un autre équipage remonte votre flotte sans jamais vous parler.
- **Point de départ** : la procédure de J3, qui décrivait un service sur une machine, et les six pannes de la phase 9.
- **Qui pilote** : le membre d'équipe en astreinte, celui de la livraison en relecture.
- **Terminé quand** : quelqu'un qui n'a pas écrit le document s'en sert de bout en bout, et on a noté où il a bloqué.

Avant d'attaquer : écrivez ce document pour l'équipage qui s'en servira, jamais pour vous-même. Écrit pour vous-même, il ne vaut rien.

La procédure de J3 décrivait un service sur une machine. Celle d'aujourd'hui décrit une flotte, et elle gagne trois sections :

- **Remonter la flotte de zéro sur une autre machine.** Depuis un poste vierge, avec le repo et rien d'autre. C'est la section que le changement de machine de production testera pour de vrai.
- **Les six pannes**, chacune avec son symptôme au tableau, sa cause, sa manœuvre, et le temps que la réparation a pris chez vous.
- **Ce qu'on regarde en premier quand un carré s'éteint.** Trois commandes, dans l'ordre, avec ce que chacune permet d'écarter. Pas dix : trois, celles qu'on tape à 3h du matin sans réfléchir.

Le test de la procédure est celui de J3 et il n'a pas d'alternative : quelqu'un qui n'a pas écrit le document s'en sert, sans poser de question à l'auteur, et on note où il bloque.

Palier 6 : le tableau de bord qui explique un carré éteint

Le tableau de la classe constate, il ne diagnostique rien : c'est un écran de score, pas un outil d'exploitation. Ce palier construit ce qui manque à côté.

Phase 11 : la mesure branchée sur la flotte

- **Objectif** : récolter des métriques qui viennent de tous les services de la flotte, et plus d'un seul.
- **Point de départ** : Prometheus et Grafana montés en J3, branchés sur une cible unique.
- **Qui pilote** : le membre d'équipe en charge de la mesure.
- **Terminé quand** : chaque service apparaît comme cible dans Prometheus, et le tableau de bord est exporté en JSON dans le repo.

Le repo d'abord : exportez la définition du tableau de bord en JSON, et committez-la. Un dashboard qui n'existe que dans un navigateur n'existe pas.

[Prometheus](#) et [Grafana](#), montés en J3, se rebranchent sur la flotte. Ce qui change, c'est le nombre de cibles : chaque service expose ses propres métriques, et la configuration de Prometheus les liste toutes.

Les métriques qui comptent aujourd'hui, en plus de celles de J3 :

- le **nombre de coups encaissés** par service et par seconde,
- la **durée de la route** `/travail`, en histogramme, parce que la moyenne masque exactement ce qu'on cherche,

- le **nombre d'exemplaires** qui répondent réellement pour chaque service,
- l'**état de la dépendance** de chaque service, en un simple 0 ou 1.

Phase 12 : les quatre panneaux qui suffisent à raconter

- **Objectif** : nommer une panne en regardant un écran, sans ouvrir un terminal.
- **Point de départ** : les métriques de la phase 11, et un tableau de bord qui affiche tout sans rien trancher.
- **Qui pilote** : le membre d'équipe en charge de la mesure et celui en astreinte, ensemble.
- **Terminé quand** : quelqu'un de l'équipage nomme une panne tirée au sort en ne regardant que les quatre panneaux.

Réflexe commit : ajoutez un panneau, exportez, committez. C'est ce qui permet de comparer deux versions du dashboard en fin de journée.

L'exercice n'est pas d'ajouter des panneaux, il est d'en avoir peu et de savoir ce que chacun élimine. Quatre questions, dix secondes chacune, pendant que quelqu'un parle devant la classe :

1. *Est-ce que le service reçoit du trafic ?* Un carré éteint avec du trafic qui arrive et une charge qui monte ne raconte pas la même histoire qu'un carré éteint avec une courbe à plat.
2. *Est-ce qu'il répond, et en combien de temps ?* La latence qui explose avant l'extinction désigne la saturation ; l'extinction sans montée de latence désigne autre chose.
3. *Est-ce que ses dépendances vont bien ?* La panne 2 et la panne 6 se distinguent ici, et nulle part ailleurs.
4. *Est-ce que la version qui tourne est celle qu'on croit ?* La panne 5 se voit ici en deux secondes.

L'épreuve du palier, plus dure qu'elle n'en a l'air : quelqu'un tire une panne pendant qu'un autre membre de l'équipage regarde uniquement le tableau de bord, sans terminal et sans savoir ce qui a été tiré. Il doit nommer la panne. Si les quatre panneaux ne suffisent pas, il en manque un, ou il y en a trop.

Palier 7 : porter la flotte sur le cluster

La flotte tient sur une machine, et cette machine est un point de défaillance unique : elle tombe, tous les carrés s'éteignent ensemble. Le cluster de J4 supprime cette dépendance, et il donne au passage la seule façon propre de livrer une version pendant que la classe tire sur vos carrés.

Phase 13 : les manifestes de la flotte

- **Objectif** : faire tourner la flotte sur le cluster, sans plus dépendre du poste d'une seule personne.
- **Point de départ** : le `compose.prod.yml` qui décrit la flotte, le cluster `todo-cluster` de J4, et le volume du pavillon.
- **Qui pilote** : le membre d'équipe en charge de la livraison, celui de l'état pour le stockage.
- **Terminé quand** : les mêmes carrés s'allument depuis le cluster, pavillon compris, avec un fichier par objet dans le repo.

Avant d'écrire un manifeste : rangez un fichier par objet dans un dossier dédié, et faites un commit par service porté. Mélanger les deux mondes dans un même commit rend le retour arrière impossible.

Le cluster `todo-cluster` de J4 accueille la flotte. Chaque service devient un Deployment et un Service, la base garde son PersistentVolumeClaim, les variables passent par ConfigMap et Secret, et un Ingress expose le front.

Le pavillon mérite une attention particulière : le volume Docker de ce matin devient une PVC, et la question de savoir qui la monte se pose autrement. Deux exemplaires d'un même service qui montent le même volume écrivent dans le même fichier, ce qui marche pour un pavillon et ne marcherait pas pour tout.

Phase 14 : la mise à jour pendant le feu

- **Objectif** : livrer une nouvelle version pendant que la classe tire, sans qu'un seul carré s'éteigne.
- **Point de départ** : la flotte portée sur le cluster en phase 13, et le rolling update de J4.
- **Qui pilote** : le membre d'équipe en charge de la livraison, celui en astreinte au chronomètre.
- **Terminé quand** : le tag change sur les carrés sans extinction ni retard, et les deux lignes du dernier relevé sont remplies.

Dernier rappel sur le repo : notez dans le carnet de la flotte le chiffre qui sort de cette phase. C'est la démonstration la plus visible de la journée.

Voilà le geste que le palier prépare : **livrer une nouvelle version pendant que la classe tire sur vos carrés, sans qu'aucun ne s'éteigne**. Le rolling update de J4, avec ses `maxSurge` et `maxUnavailable`, plus les deux sondes qui décident quel exemplaire reçoit du trafic.

Bien câblé, le tag d'image change sur le carré, le carré ne s'éteint pas, et le retard ne monte pas. Mal câblé, le carré pâlit quelques secondes pendant que du trafic part vers un exemplaire qui n'est pas prêt à le recevoir.

Le dernier relevé du carnet, à faire deux fois pour pouvoir comparer :

La façon de livrer	Carrés éteints pendant la livraison	Coups perdus	Durée totale
<code>docker compose up -d</code> sur la machine cible			
Rolling update sur le cluster			

Combien de coups une livraison coûte-t-elle vraiment ? Personne dans la salle n'a la réponse avant de l'avoir mesurée, et l'écart entre les deux lignes est le résumé le plus court de la semaine.

6 - L'ouverture du feu et les passages

L'essentiel du déroulé

À heure fixe, quel que soit l'état d'avancement de chacun, le feu s'ouvre. Tous les carrés de la classe deviennent cliquables, par tout le monde, en même temps. Un quart d'heure de chaos collectif, où chacun découvre ce que sa flotte encaisse et à partir de quand elle pâlit.

Personne n'est exclu de ce moment. Un équipage qui n'a qu'un carré allumé participe avec un carré, et il apprendra la même chose que les autres, en pire et donc plus vite.

Viennent ensuite les passages, dix minutes par équipage :

1. **La flotte, en deux minutes**. Ce que fait l'application, combien de carrés, et pourquoi ce pavillon.
2. **La livraison, en direct**. Un `git push`, et la classe regarde les tags changer sur vos carrés.

3. **La panne, tirée au sort.** Quelqu'un de l'équipage lance le script, personne ne sait ce qui sort.
4. **Le diagnostic, à voix haute,** avec le tableau de bord projeté à côté du tableau de la classe.

La minute d'hypothèse

Au moment où la panne est lancée, toute la classe a **soixante secondes pour écrire son hypothèse**, avant que l'équipage au tableau ne dise quoi que ce soit. Une ligne suffit : "la base est tombée", "le tag n'existe pas", "le service tourne mais il ne joint plus le tableau". L'équipage annonce la sienne ensuite, et on découvre ensemble qui avait raison.

Le rituel oblige chacun à lire un tableau de bord qui n'est pas le sien, ce qui est exactement l'exercice d'une astreinte prise sur un service qu'on n'a pas écrit.

Ce qui compte pendant le passage

Pas la vitesse de réparation. Sérieusement, pas la vitesse.

Ce qui compte, c'est ce qui est dit à voix haute pendant que ça se passe : nommer ce qu'on voit, dire ce qu'on élimine et pourquoi, annoncer la manœuvre avant de la lancer, reconnaître quand on ne sait pas. Un équipage qui répare en trente secondes sans savoir dire ce qui s'est passé réussit moins bien son passage qu'un équipage qui n'a pas réparé mais qui a raconté correctement ce que son tableau de bord montrait.

Même critère qu'en J3 pendant la passation d'astreinte, et ce n'est pas une coïncidence : c'est le métier.

7 - Ce qui est rendu

Le repo de groupe, public ou avec accès donné, contient à la fin de la journée :

- le code des trois services au minimum, plus la base, chacun avec son propre `Dockerfile`,
- le `compose.prod.yml` qui décrit la flotte, et les manifestes du cluster pour les équipages qui l'ont portée,
- les workflows dans `.github/workflows/`, dont le job qui livre la flotte entière,
- `pannes.sh` et le tableau des six pannes, complété avec ce que le tableau a montré pour chacune,
- la définition du tableau de bord, exportée en JSON,
- `docs/RUNBOOK_FLOTTE.md`, avec la remontée depuis zéro sur une autre machine,
- un `README.md` qui donne le nom de l'équipage, sa couleur, son pavillon, le nombre de carrés jurés, et qui tient quel rôle,
- le **carnet de la flotte**, avec ses relevés chiffrés, y compris ceux qui sont décevants.

Aucun mot de passe, aucune clé privée, aucun fichier d'environnement réel dans le repo. Un `git log -p` qui les ferait apparaître, même supprimés depuis, compte comme une fuite.

Grille d'évaluation :

Critère	Poids	Ce qui est regardé
Conteneurisation de la flotte	20 %	une image par service, multi-stage, non-root, healthcheck, rien d'inutile dedans

Critère	Poids	Ce qui est regardé
Déploiement automatisé	20 %	un push livre la flotte entière, le job échoue si un service ne répond pas, retour arrière possible
Surveillance	15 %	métriques exposées par service, quatre panneaux qui permettent de nommer une panne sans terminal
Runbook et pannes	15 %	six pannes documentées avec symptôme, cause et manœuvre, remontée depuis zéro éprouvée par un tiers
La flotte tenue	15 %	carrés jurés contre carrés tenus pendant l'ouverture du feu, saturation encaissée
L'état qui survit	10 %	pavillon dans un volume, redéploiement sans perte, base qui garde ses données
Rigueur du repo	5 %	commits atomiques, pull requests relues par un autre rôle, aucun secret versionné

Apprécié en plus, sans être attendu : un service annexe qui fait quelque chose que personne d'autre n'a eu l'idée de faire, une dégradation gracieuse visible au tableau pendant la panne 2, une livraison faite pendant l'ouverture du feu sans qu'un carré s'éteigne, un panneau de tableau de bord qui a permis de nommer une panne avant l'équipage qui la subissait.

Récapitulatif Jour 5

Ce qu'on a appris cette semaine

1. **Un conteneur n'est pas une petite machine virtuelle** : des namespaces, des cgroups, et un système de fichiers en couches, montés à la main en J1 avant qu'un seul `docker run` ne soit tapé.
2. **Une image de production se construit, elle ne s'improvise pas** : multi-stage, utilisateur non privilégié, sonde de santé, et une taille qu'on mesure avant et après chaque décision.
3. **Une pipeline vérifie avant de livrer, et elle sait s'arrêter** : lint, tests, image construite une seule fois et promue telle quelle du staging à la production.
4. **Déployer, c'est remplacer une version par une autre sans casser ce qui tourne** : idempotence, retour arrière chronométré, et un job qui échoue proprement plutôt qu'une pipeline verte devant un service mort.
5. **On ne surveille pas pour surveiller** : quatre panneaux qui permettent de nommer une panne valent mieux que quinze qui décorent, et une métrique se choisit par la question à laquelle elle répond.
6. **Le cluster maintient un état voulu, il ne devine rien** : ce qu'il voit, il le répare tout seul ; ce qu'on lui cache reste cassé jusqu'à ce qu'une main humaine intervienne.
7. **Une procédure vaut ce que vaut la personne qui n'a pas participé à l'écrire** : c'est le seul test, et il est passé deux fois cette semaine.
8. **Un service qui va bien et un service qui arrive à dire qu'il va bien sont deux choses différentes** : la sixième panne de la Bataille Navale, et la moitié des fausses alertes en production.

Où continuer après le cours

- **Refaites la flotte tout seul, en une soirée.** Trois services, une pipeline, un tableau de bord. Ce qui a pris une journée à quatre en prend trois heures à un, et c'est le meilleur test de ce qui est vraiment acquis.
- **Regardez ce que fait votre entreprise ou votre lieu de stage** : où sont les images, qui a le droit de déployer, combien de temps prend un retour arrière, et qui reçoit l'alerte à 3h du matin. Les mêmes questions, en plus gros.
- [La documentation Docker sur les bonnes pratiques d'images](#) et [le guide Prometheus sur le choix des métriques](#) sont les deux lectures qui rapportent le plus vite après cette semaine.
- **Le cluster ne s'arrête pas à ce qu'on en a vu** : autoscaling, gestion des ressources, [Helm](#) pour empaqueter des manifestes qui se ressemblent tous. Le modèle déclaratif de J4 est la clé qui ouvre tout le reste.

Bravo pour cette semaine. Lundi, la question était "c'est quoi un conteneur" ; vendredi, votre équipage a tenu une flotte en vie devant toute la classe, sous la charge et sous la panne. 🔥